CLOUD ARCHITECTURE PROJECT PRESENTATION

Highly Available Web Application Architecture

Azure Vnet

Load Balancer

Linux VMs

Azure SQL (Private)

DNS ZONE

INTRODUCTION

Project Overview

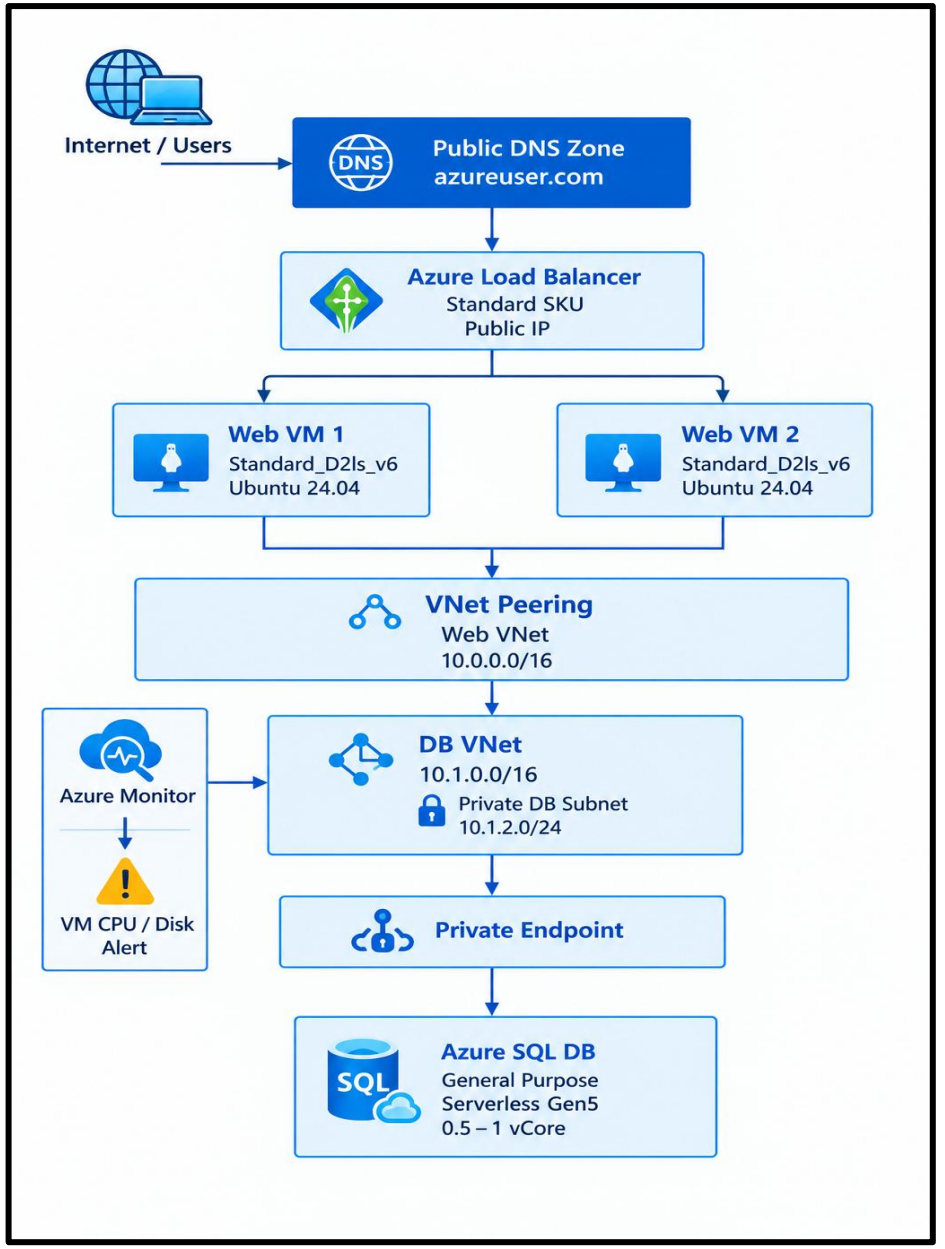
- Deploys a production-style web application on Microsoft Azure using industry-standard cloud architecture patterns
- Solves the single-point-of-failure problem using two web servers behind a load balancer
- Keeps the database completely private — it is never reachable directly from the internet
- Uses DNS so users access the application through a friendly domain name, not an IP address
- Demonstrates cloud portability by mapping the identical design onto AWS services

2
Web VMs for High Availability

0
Public Endpoints on the Database

1
Architecture, Two Clouds (Azure + AWS)

Architeture Diagram

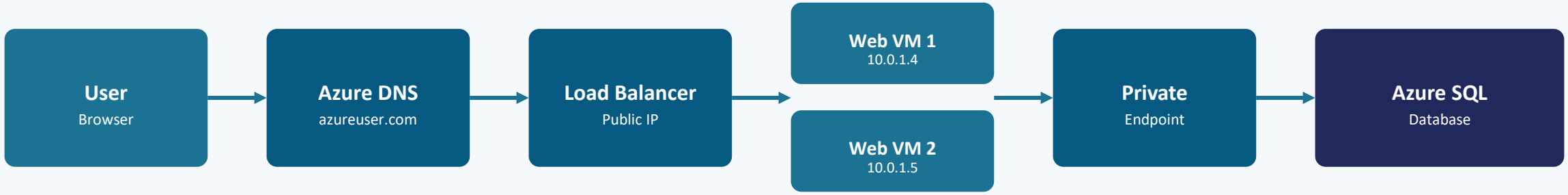


PLANNING

Requirements

1. **Host a Web Application**
   Run the application code on reliable compute
2. **Multiple Web Servers**
   Avoid a single point of failure
3. **Load Balancing**
   Spread traffic evenly across servers
4. **Secure DB Connectivity**
   Encrypted, controlled access to data
5. **Private Database Access**
   No direct internet exposure
6. **Monitoring**
   Visibility into server health
7. **Scalability**
   Room to grow with demand
8. **DNS-Based Access**
   A friendly, memorable domain name

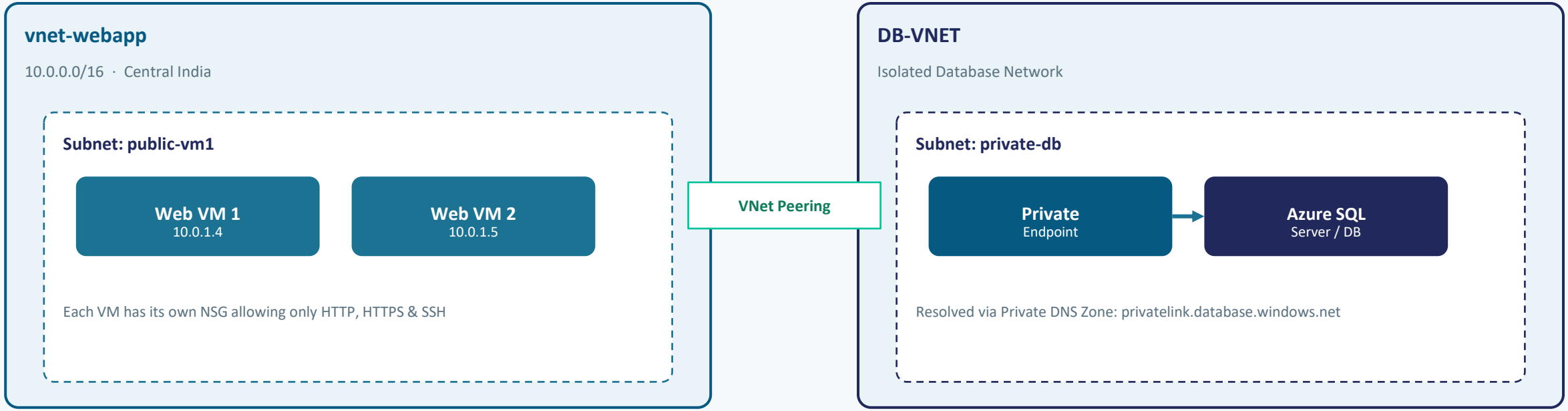
Work Flow



- The user's browser resolves the domain name through Azure DNS, which returns the load balancer's public IP
- The Load Balancer receives the request and forwards it to whichever web VM is healthy and available
- Web VM 1 and Web VM 2 run the identical application, registered in the same backend pool
- The web VM reaches the database only through a Private Endpoint over Azure's internal network — never over the public internet
- The Azure SQL Database processes the query and the response travels back the same private path

ARCHITECTURE

Network Architecture

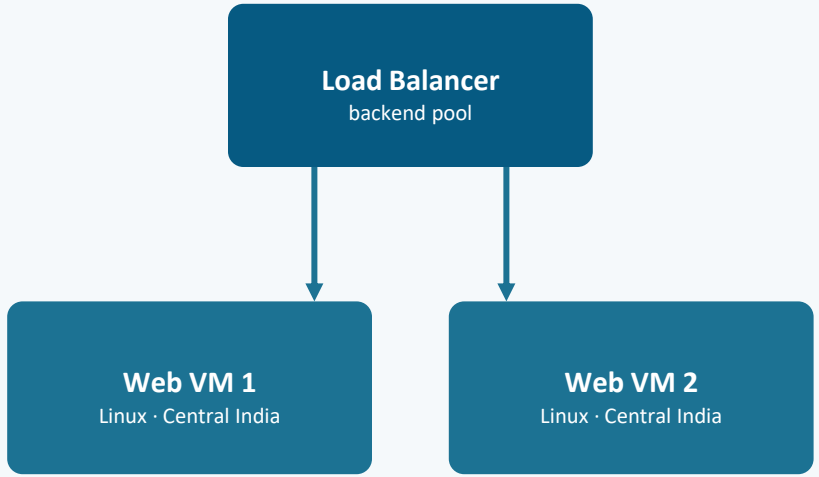


- Two separate VNets isolate the web layer from the database layer at the network level
- VNet Peering privately connects vnet-webapp and DB-VNET without routing through the public internet
- The Private Endpoint inside DB-VNET gives the SQL Database a private IP address reachable only from peered networks
- A Private DNS Zone makes the database's normal hostname resolve to that private IP automatically

ARCHITECTURE

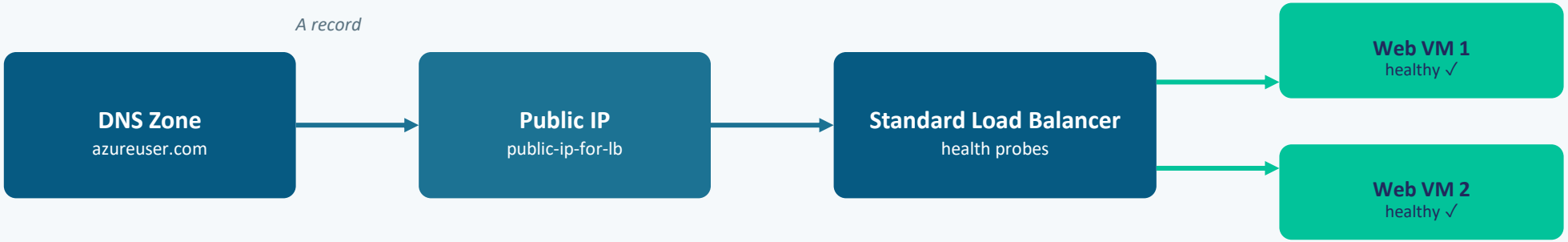
Web Layer

- Two identical Linux virtual machines — web-vm1 and web-vm2 — host the web application
- Why two servers? If one VM fails or is restarted, the other keeps serving users with zero downtime
- Both VMs are registered in the same Load Balancer backend pool, so traffic is spread across them
- Each VM has its own Network Security Group, allowing only HTTP (80), HTTPS (443) and SSH (22)
- This design is what makes the application highly available rather than dependent on a single machine



ARCHITECTURE

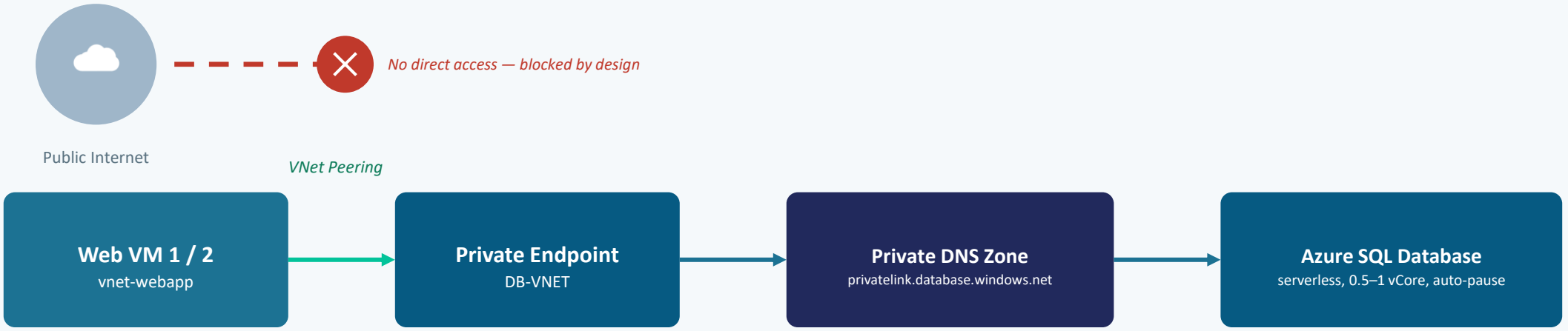
Load Balancer & DNS



- The Azure DNS Zone maps the human-friendly domain to the load balancer's Public IP using an A record
- The Standard Load Balancer is the single public entry point — it distributes every request across the backend VMs
- Continuous health checks (probes) confirm each VM is responding correctly before sending it traffic
- If a VM fails a health check, the Load Balancer automatically stops routing traffic to it
- Together, DNS and the Load Balancer are what let users type a simple web address and always reach a healthy server

ARCHITECTURE

Database & Security



- Azure SQL Database runs on the serverless tier, auto-pausing when idle to save cost, and scaling to ~0.5–1 vCore under load
- The database has no public endpoint — it is reachable only through the Private Endpoint inside DB-VNET
- Network Security Groups and private-only connectivity together stop any direct external access to data
- The application connects using a normal SQL connection string, but that traffic never leaves Azure's private backbone

Data Base we are Connected

azureuser@web-vm1: ~

```
-----------------------------------------------
sql-db


(1 rows affected)
1> SELECT * FROM Students;
2> SELECT * FROM Students;

Sqlcmd: Warning: The last operation was terminated because the user pressed CTRL
+C.
azureuser@web-vm1:~$ SELECT * FROM Students;
GO
SELECT: command not found
GO: command not found
azureuser@web-vm1:~$ /opt/mssql-tools/bin/sqlcmd -S sql-server-4.database.window
s.net -d sql-db -U database -P '<YOUR_PASSWORD>'
Sqlcmd: Error: Microsoft ODBC Driver 17 for SQL Server : Login failed for user '
database'..
azureuser@web-vm1:~$ /opt/mssql-tools/bin/sqlcmd -S sql-server-4.database.window
s.net -d sql-db -U database -P 'Ganesha@1234'
1> SELECT * FROM Students;
2> GO
StudentID   Name
                              Course
                                                    Marks
----------- ------------------------------------------------------------------
------------------------------ --------------------------------------------------
-------------------------------------------------- -----------
          1 Rahul
                              MBA
                                                         75
          2 Priya
                              MBA
                                                         82
          3 Arun
                              MCA
                                                         68
          4 Kiran
                              MBA
                                                         78
          5 Sneha
                              MBA
                                                         88
          6 Vijay
                              MCA
                                                         72

(6 rows affected)
1>
```

OPERATIONS

Monitoring

